

HolyGATC / GATC46

An educational manual for a genome-sequence compiler and stack virtual machine. Version covered: GATC46.py 1.1.0, repository 23x41/DNAcompiler.

This is recreational systems programming. The genetic code is used as a naming scheme and an encoding. It is not a biology protocol, and a .dna file from this compiler is not a laboratory sequence.

What you are learning

A compiler reads text a person wrote and turns it into a smaller form a machine can execute. An interpreter or virtual machine then walks that form one instruction at a time. HolyGATC does both jobs in one Python file.

The machine underneath is a stack computer. There are no named registers. Values sit on a pile. The top of the pile is called TOS. Most instructions take their inputs from the top and leave their result there. Forth programmers write the effect as (before -- after). Example: ADD is (a b -- a+b). That means it pops b, pops a, and pushes a+b.

Three representations of the same program are the point of the project:

- Amino-acid assembly (.aa). Letters and words a person writes. K is LIT, A is ADD, H is HALT.
- Bytecode (.gbc). One opcode byte, plus a 4-byte little-endian immediate when the opcode needs one. LIT is the only opcode with an immediate.
- Packed DNA (.dna). That bytecode wrapped as a FASTA record: ATG, a 16-bit length, the payload in base-4, then TAA.

A fourth path, gene mode, does not store bytecode. It pretends to be a ribosome: find ATG, read codons, translate them with the standard genetic code, and treat the amino-acid letter as an opcode.

1. How to run it

Requirements

Python 3. The compiler is one file, GATC46.py. No packages are required. From the directory that contains the file:

```
python3 GATC46.py
python3 GATC46.py version
python3 GATC46.py opcodes
```

The first form prints the banner and help. version prints HolyGATC 1.1.0. opcodes prints the 46-row table.

Commands

The first argument is the command. A filename alone is not enough. That is why 'python3 GATC46.py newops.aa --dna -o newops.dna' fails: the command name compile is missing.

```
python3 GATC46.py run program.aa --no-banner
python3 GATC46.py disasm program.aa
python3 GATC46.py compile program.aa -o program.gbc
python3 GATC46.py compile program.aa --dna -o program.dna
python3 GATC46.py run program.dna --no-banner
python3 GATC46.py translate program.dna
python3 GATC46.py run --gene gene.dna --no-banner
python3 GATC46.py disasm --gene gene.dna
- run FILE: assemble or decode, then execute. A banner is printed to stderr unless
  --no-banner is set.
```

- compile FILE -o OUT: write raw bytecode. Add --dna to write a FASTA genome instead.
- disasm FILE: print a numbered instruction listing. Addresses are instruction slots, not byte offsets.
- translate FILE: DNA bases to amino-acid letters, from the first ATG to a stop codon.
- --gene: treat the file as a coding sequence, not as packed bytecode.
- --max-steps N: stop a runaway loop. Default is 1,000,000 steps.

Source suffixes .aa are assembly. Suffixes .dna, .gatc, .fa, .fasta, and .fna are treated as DNA. A .gbc file is raw bytecode; a file whose first four bytes are GATC is also recognized as a container.

2. Assembly language

Lexing rules

Whitespace is ignored. A # starts a comment that runs to the end of the line. A label is :NAME and is stored uppercase. A bare name that is not an opcode pushes that label's address, which is the same as writing K NAME.

Numbers are decimal, negative decimal, or hex (0x1A). A bare number is a LIT. A minus followed by a digit is a negative literal. A bare minus is the DEC opcode. Strings use double quotes. Escapes are \n, \t, \r, \, and \". A string does not push text. It emits, for each character, LIT of that character's code, then PUTC.

An opcode may be written as its one-character symbol or as its word. Word aliases include ADD, PUSH and LIT, HALT, PRINT for PUTN, EMIT for PUTC, and LOAD for FETCH. Only LIT takes an operand in the source. Jumps do not. You push the address, then execute B, J, or *.

Control shape

Labels are instruction indexes after assembly, counting expanded string instructions. Subroutines must sit after a jump to main, or execution falls into them.

```
K MAIN
B
:DOUBLE
D A
R
:MAIN
K 21
K DOUBLE
C
Y
H
```

Reading it: push the address of MAIN and jump there. Later, push 21, push the address of DOUBLE, and call it. DOUBLE duplicates the top and adds, so 21 becomes 42, then returns. Y prints 42. H stops.

A complete program

Read one integer. Print n*n and n+n. Input 7 produces: digit? n*n=49 n+n=14

```
"digit? "
I
D
D
M
Z
D
A
```

```

Z
" n*n="
Y
" n+n="
Y
K 10
W
H

```

Stack for input 7, rightmost is top. I leaves [7]. Two D instructions leave [7, 7, 7]. M multiplies the top pair: [7, 49]. Z swaps: [49, 7]. D copies the 7: [49, 7, 7]. A adds the top pair: [49, 14]. Z puts the product on top: [14, 49]. The first Y prints 49. The second Y prints 14. K 10 W writes a newline. H halts.

The extra swaps exist because MUL and ADD both consume the top two values. One copy of n has to survive underneath the product, then be brought back to the top so it can be added to itself.

3. The 46 opcodes

Forty-six opcodes, one for each human chromosome, is the project's framing. The first 36 match the older HolyGATC set. The last ten use symbols that are not amino-acid letters, so gene mode cannot emit them. In the table, Sym is what you write, Byte is the bytecode value, and Imm means a 4-byte immediate follows the opcode.

Stack notation: a is deeper, b is on top, unless three values are shown. Flag is 1 or 0. Division is integer division.

Arithmetic

A ADD 0x01 (a b -- a+b)

Pop b, pop a, push a+b. No overflow trap; Python integers are unbounded, so the VM does not wrap. Use it for sums and, with DUP, for doubling: D A turns n into n+n.

S SUB 0x13 (a b -- a-b)

Pop b, pop a, push a-b. Order matters. The deeper value is the minuend. To compute 10-3, push 10, push 3, then S.

M MUL 0x0D (a b -- a*b)

Pop b, pop a, push a*b. D M squares the top value. Gene mode spells MUL as methionine, whose only codon is ATG, which is also the start codon. After the skipped start codon, a later ATG is multiply.

V DIV 0x16 (a b -- a/b)

Integer divide, a//b. Division by zero is a GATCError. The remainder is discarded. Negative division follows Python's floor rule.

% MOD 0x1C (a b -- a%b)

Remainder of a divided by b. Modulo by zero is an error. Useful for even/odd tests: K 2 % then N, or compare the remainder with zero.

~ NEG 0x1D (a -- -a)

Replace the top with its negation. Assembly symbol is tilde, not a letter, so gene mode cannot emit it.

+ INC 0x23 (a -- a+1)

Add one to the top. Shorter than K 1 A.

- DEC 0x24 (a -- a-1)

Subtract one from the top. A minus glued to a digit is a negative literal, not DEC. Write

the opcode as a separate token.

. ABS 0x2D (a -- |a|)

Replace the top with its absolute value. Assembly-only. Word form: ABS.

Comparison and logic

Comparisons push 1 for true and 0 for false. Bitwise operations use Python's arbitrary-precision integers. NOT is logical, not bitwise: it is 1 only when the top is exactly zero.

E EQ 0x05 (a b -- flag)

1 if a equals b, else 0.

L LT 0x0C (a b -- flag)

1 if a < b. Deeper value is the left operand.

G GT 0x07 (a b -- flag)

1 if a > b.

= NE 0x28 (a b -- flag)

1 if a != b. Assembly-only. Word form: NE.

/ GE 0x29 (a b -- flag)

1 if a >= b. Assembly-only. The slash is not DIV. DIV is V.

^ LE 0x2A (a b -- flag)

1 if a <= b. Assembly-only.

| MAX 0x2B (a b -- max)

Push the greater of the two popped values. Assembly-only.

; MIN 0x2C (a b -- min)

Push the lesser of the two. Assembly-only.

N NOT 0x0E (a -- flag)

1 if a is 0, else 0. This is the flag inverter for JZ and JNZ.

P AND 0x10 (a b -- a&b)

Bitwise and.

O OR 0x0F (a b -- a|b)

Bitwise or. O is a synthetic letter, not an amino acid, so gene mode cannot emit it.

X XOR 0x18 (a b -- a^b)

Bitwise exclusive or. Also synthetic, so assembly-only in gene mode.

< SHL 0x20 (a b -- a<<b)

Shift a left by b bits. The shift count is masked with 31, so only the low 5 bits count.

> SHR 0x21 (a b -- a>>b)

Arithmetic right shift of a by b, again masked with 31.

Stack manipulation

D DUP 0x04 (a -- a a)

Copy the top. Underflow is an error. This is how you keep a value and also operate on it.

Z SWAP 0x1A (a b -- b a)

Exchange the top two. Synthetic letter. Needed whenever an operation would consume a value you still need underneath.

U OVER 0x15 (a b -- a b a)

Copy the second value onto the top. Synthetic letter.

! DROP 0x1E (a --)

Discard the top. Symbol is an exclamation mark.

& NIP 0x27 (a b -- b)

Drop the second value and keep the top. Assembly-only. Word form: NIP.

@ ROT 0x26 (a b c -- b c a)

Rotate the top three: the deepest of the three becomes the new top. Assembly-only. Word form: ROT.

, DEPTH 0x2E (-- n)

Push the number of values currently on the stack, not counting the count itself. Empty stack pushes 0. Assembly-only. Word form: DEPTH. It does not read memory and does not consume an argument.

_ NOP 0x1B (--)

Do nothing and advance the program counter. Packed DNA spelling of the single opcode byte 0x1B is ACGT, because 00011011 groups as 00 01 10 11 and those pairs are A, C, G, T. There is no gene-mode codon for NOP.

Literals, memory, and random

K LIT 0x0B imm (-- n)

The only instruction with an immediate. In assembly, K 7 or a bare 7 pushes 7. In bytecode the opcode byte is followed by the value as 4 little-endian bytes. In gene mode, lysine (AAA or AAG) means LIT, and the next codon is the number, base 4, range 0 to 63. A=0, C=1, G=2, T=3. GGA is $2*16+2*4+0 = 40$.

T STORE 0x14 (val addr --)

Write val into memory[addr]. Address is on top. Memory is 4096 cells, indexed from 0. Out of range is an error.

F FETCH 0x06 (addr -- val)

Replace an address with memory[addr]. Word alias: LOAD. Cells start at 0.

\$ RAND 0x22 (n -- r)

Push a random integer in [0, n). If n is less than or equal to 0, push 0. Uses Python's random.randrange.

Control

B JMP 0x02 (addr --)

Pop an address and set the program counter to it. Addresses are instruction indexes. Typical source shape is K LABEL then B. B is synthetic, not an amino acid.

J JZ 0x0A (flag addr --)

Pop the address, pop the flag. Jump if the flag is 0. Otherwise go to the next instruction. Typical shape: push the condition, push the label, then J.

*** JNZ 0x25 (flag addr --)**

Jump if the flag is not 0. Assembly-only. Word form: JNZ.

C CALL 0x03 (addr --)

Push the return address on a separate call stack, then jump. The return address is the instruction after the CALL. Call depth limit is 4096.

R RET 0x12 (--)

Pop the call stack into the program counter. RET without a CALL is an error. Gene mode can emit this: arginine codons CGT, CGC, CGA, CGG, AGA, AGG.

H HALT 0x08 (--)

Stop with exit code 0. In gene mode, TAA, TAG, and TGA become HALT and end translation.

Q QUIT 0x11 (code --)

Stop and use the popped value as the process status. Gene mode can emit it from glutamine, CAA or CAG.

Input and output

Y PUTN 0x19 (n --)

Write the top as a decimal numeral, with no added space or newline. Word alias: PRINT. Gene letter Y is tyrosine, TAT or TAC.

W PUTC 0x17 (n --)

Write the low 8 bits of the top as one character. 10 is newline, 32 is space. Word alias: EMIT. A quoted string is sugar for a run of LIT/PUTC pairs.

I IN 0x09 (-- n)

Read a line and push int(line). Empty input pushes -1. A line that is not an integer pushes 0. Gene letter I is isoleucine, ATT, ATC, or ATA.

? GETC 0x1F (-- n)

Read one character and push its code. End of file pushes -1. Symbol is a question mark, so assembly-only.

4. Packed DNA files

compile --dna does not translate through the genetic code. It packs the bytecode so the file looks like a tiny genome.

The record is ATG, then a 16-bit big-endian length, then the payload, then TAA. ATG and TAA here are framing, not a start codon and a halt opcode. The length counts payload bytes, not bases. FASTA wraps at 60 characters and the header reads HolyGATC genome from <file>.

Each payload byte becomes four bases, high bits first. A=00, C=01, G=10, T=11. So one byte is always four letters, and the whole payload length in bases is four times the byte length, plus eight bases for the length word.

NOP is 0x1B = 00011011 = ACGT. HALT is 0x08 = 00001000 = AAGA. ADD is 0x01 = 00000001 = AAAC. A one-instruction NOP genome is:

```
ATG AAAAAAAC ACGT TAA
```

AAAAAAAC is the length word 0x0001. ACGT is the NOP. The runner strips a leading ATG and a trailing TAA, rebuilds bytes from groups of four bases, reads the 16-bit length, and checks that the payload is exactly that long.

U in an input file is rewritten to T. Characters outside ACGT are dropped, so a FASTA header does not become code. The payload length must be a multiple of four bases or decoding fails.

5. Codons and gene mode

Gene mode is the other DNA path. You pass --gene. The compiler finds ATG, skips that start codon, and reads the rest in groups of three. Each codon is looked up in the standard genetic code. The amino-acid letter is the opcode. A stop codon, written internally as *, becomes HALT and ends the program. If translation never sees a stop, a HALT is appended.

The letter mapping used by this compiler:

- A alanine, GCT/GCC/GCA/GCG: ADD
- C cysteine, TGT/TGC: CALL
- D aspartic acid, GAT/GAC: DUP
- E glutamic acid, GAA/GAG: EQ
- F phenylalanine, TTT/TTC: FETCH
- G glycine, GGT/GGC/GGA/GGG: GT
- H histidine, CAT/CAC: HALT
- I isoleucine, ATT/ATC/ATA: IN
- K lysine, AAA/AAG: LIT, and the next codon is a 0..63 immediate
- L leucine, TTA/TTG/CTT/CTC/CTA/CTG: LT
- M methionine, ATG: MUL. The first ATG is skipped as start.
- N asparagine, AAT/AAC: NOT
- P proline, CCT/CCC/CCA/CCG: AND
- Q glutamine, CAA/CAG: QUIT
- R arginine, CGT/CGC/CGA/CGG/AGA/AGG: RET
- S serine, TCT/TCC/TCA/TCG/AGT/AGC: SUB
- T threonine, ACT/ACC/ACA/ACG: STORE
- V valine, GTT/GTC/GTA/GTG: DIV
- W tryptophan, TGG: PUTC
- Y tyrosine, TAT/TAC: PUTN

Synthetic opcodes have no codon: B JMP, J JZ, O OR, U OVER, X XOR, Z SWAP, and every symbol opcode (`_`, `%`, `~`, `!`, `?`, `<`, `>`, `$`, `+`, `-`, `*`, `@`, `&`, `=`, `/`, `^`, `|`, `;`, `:`, `.`, `,`). Gene mode cannot express those.

The classic gene that prints 42:

```
ATG AAA GGA AAA AAG GCT TAT TAA
```

Skipped start. AAA is LIT, GGA is 40. AAA is LIT, AAG is 2. GCT is alanine, ADD. TAT is tyrosine, PUTN. TAA is stop. 40+2 is printed as 42.

translate prints the amino-acid string rather than executing. For that gene it is MKAKAY*, with the start methionine included in the letter dump. Execution skips that first M.

Codon immediates are not ASCII and not bytecode. They are six bits: each base contributes two bits, high base first. AAA is 0. TTT is 63. You cannot push 100 in gene mode with one LIT. Packed DNA can, because LIT's immediate there is a full 32-bit word.

6. How GATC46.py is built

The file is about 874 lines and has no library dependencies beyond the Python standard library. Reading it is a tour of a small language implementation: tables, lexer, assembler, encoder, decoder, and a loop that executes one instruction.

Tables

OPDEF is the source of truth. Each row is symbol, name, byte, whether an immediate follows, and a stack description. From that list the program builds LETTER_TO_OP, NAME_TO_BYTE, BYTE_TO_NAME, BYTE_HAS_IMM, and LETTER_HAS_IMM. ALIASES maps words such as PRINT and EMIT onto those names. GENETIC_CODE is the NCBI-style codon table, with stop codons mapped to *. BITS_OF_BASE and BASE_OF_BITS are the 2-bit packing. Limits are MEM_SIZE 4096, STACK_LIMIT 65536, CALL_LIMIT 4096, and STEP_LIMIT_DEFAULT 1000000.

Errors and records

GATCError is the single exception type. main catches it and prints error: ... to stderr. Token holds kind, value, line, and column. Instr holds the opcode name, an optional immediate, an optional unresolved label, and a source line. Using two small records keeps

the assembler from passing raw tuples around.

Lexer: tokenize_aa

A hand-written scanner, not a regular expression. It walks the string with an index. Spaces and tabs are skipped. Newlines bump the line counter. # skips to the newline. A quote starts a string, with the five escapes, and rejects a raw newline inside the quotes. A colon starts a label, which is forced to uppercase. A digit, or a minus followed by a digit, starts a number; 0x switches to hex. A single character that is an opcode and is not followed by a letter is an opcode token. A longer word is an alias if it is in ALIASES, otherwise an identifier, which later means a label address. Anything else is an error with line and column.

Assembler: assemble_tokens and resolve

Two passes, in the classic way. Pass one walks tokens. A label records the current instruction count and emits nothing. A string emits LIT plus PUTC per character, which is why disassembly of a short string is long. A number emits LIT. An identifier emits LIT with the label still unresolved. An opcode emits one Instr, and if it has an immediate the next token must be a number or an identifier. Pass two, resolve, replaces each label name with the instruction index stored in the label table. A missing label is an error. Duplicate labels are an error in pass one.

Bytecode: encode_bytecode and decode_bytecode

Encoding writes the opcode byte. If that byte has an immediate, it writes the integer as four little-endian bytes, masked to 32 bits. Decoding reverses that. This is why disassembly addresses and bytecode offsets differ: a LIT occupies five bytes and one instruction slot. The runner also accepts a .gbc whose first four bytes are the ASCII magic GATC.

DNA codec

strip_to_bases uppercases, turns U into T, and keeps only ACGT. bytecode_to_dna builds a packet of a 2-byte big-endian length plus the bytecode, expands each byte to four bases, and wraps ATG and TAA. dna_to_bytecode strips framing, requires a multiple of four bases, rebuilds bytes, checks the length, and returns the payload. codon_value packs three bases into a 0..63 integer for gene-mode literals. compile_gene is the ribosome loop described in section 5. compile_source guesses the input: a DNA suffix, or a string that is only bases and starts with ATG and is long enough, goes through the DNA decoder. Otherwise it tokenizes and assembles.

FASTA, translate, disasm

write_fasta emits a > header and wraps at 60. translate_coding_sequence is the letter dump used by the translate command. disasm prints each instruction as index, name, and immediate if present. It does not try to recover string quotes. A printed word is a run of LIT/PUTC pairs.

The virtual machine

VM.__init__ holds the program, a program counter, a data stack, a call stack, a 4096-cell memory list, stdin, stdout, a halted flag, an exit code, and a step count. push and pop enforce the stack limit and underflow. run is the fetch loop: check the step budget, check that the program counter is in range, fetch the Instr, count the step, call _exec. _exec is a long chain of opcode names. Arithmetic pops b then a so the deeper value stays the left operand. Jumps assign the program counter instead of incrementing it. CALL saves pc+1. RET restores it. IN uses readline and int(). GETC uses read(1). PUTN and PUTC write and flush. HALT and QUIT set halted. An unknown name is an error, which should be unreachable if the

assembler is the only producer.

The command line

main builds an argparse parser. The command is optional and defaults to help, chosen from run, compile, disasm, translate, opcodes, help, and version. Flags are -o, --dna, --max-steps, --no-banner, and --gene. parse_intermixed_args lets flags sit among positional arguments. help and version return before opening a file. opcodes prints OPDEF. Every file command reads the source as text, then branches. compile always encodes bytecode and, with --dna, wraps it in FASTA. disasm either gene-compiles, decodes a .gbc, or assembles. run does the same selection and then VM.run. GATCError becomes a nonzero exit.

7. Pitfalls and exercises

- The command name is required. compile, run, and disasm come before the filename.
- JZ jumps when the flag is 0, not when it is 1. The address is on top of the flag.
- Subroutines placed before main will run as the program falls into them.
- Strings expand. A label after a string is not at the source-line count.
- Gene mode immediates only reach 63, and cannot encode JMP, JZ, SWAP, or the symbol opcodes.
- IN on a blank or non-numeric line pushes 0, which is indistinguishable from the user typing 0. End of file is -1.
- PUTN does not print a newline. Emit 10 yourself with K 10 W.
- Memory is 4096 cells. The data stack and the call stack are separate.

Exercises

- Print the numbers 1 through 5 with a counted loop. You need a label, DUP, a comparison, JZ or JNZ, and INC.
- Write DOUBLE and SQUARE as subroutines and call both on one input.
- Store the input at address 0, then fetch it twice, so you do not need the swap dance.
- Compile the same program both --dna and as assembly, disassemble both, and confirm the instruction list matches.
- Hand-encode HALT as packed DNA and run the FASTA file.

Source of the implementation: GATC46.py in github.com/23x41/DNAcompiler, version 1.1.0. The genetic-code table follows the standard codon table. The opcode letters are a project convention, not a biological claim.